

Course Number	718
Course Title	Embedded Systems Design
Semester/Year	Fall 2026
Instructor	Arghavan Asad
T.A	Osama Harmouche
Section Number	07

Lab 1 Report

Report Title	Introduction to Keil uVision & ARM Cortex M3
Submission Date	September 22, 2026
Due Date	September 22, 2026

Student Name	Student ID	Signature*
Ali Israr	501242427	A.I

**By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at:*
<https://www.torontomu.ca/senate/policies/academic-integrity-policy-60/>.

Introduction

This lab deals with developing an embedded application through the use of Keil uVision IDE, ARM Cortex-M3 architecture and NXP LPC1768 microcontroller with the help of MCB1700 development board. This lab highlights the relationship between the software and microcontroller's hardware peripherals such as GPIO inputs and outputs.

The primary application developed in this lab involves the use of MCB1700 board's joystick for detecting inputs from directions and showing the latest direction on LCD and LEDs. The joystick is built as a GPIO input in the form of pins P1.20, P1.23, P1.24, P1.25 and P1.26 of LPC1768. The code reads those pins and finds the selected direction from the user and gives the required LED and LCD outputs.

Code Overview

```
#include <stdio.h>
#include <stdint.h>
#include "Blinky.h"
#include "LPC17xx.h"
#include "GLCD.h"
#include "LED.h"
#include "KBD.h"

#define __FI 1
#define __USE_LCD 1

int main(void)
{
    uint32_t joystick;
    uint32_t last_direction = 0;

    LED_Init();
    KBD_Init();

#ifdef __USE_LCD

    GLCD_Init();
    GLCD_Clear(White);

    GLCD_SetBackColor(Blue);
    GLCD_SetTextColor(Yellow);

    GLCD_DisplayString(
        0,
        0,
        __FI,
        (unsigned char *) "    COE718 LAB 1    "
    );

    GLCD_SetTextColor(White);

    GLCD_DisplayString(
        1,
        0,
        __FI,
        (unsigned char *) "    JOYSTICK DEMO    "
    );

    GLCD_SetBackColor(White);
    GLCD_SetTextColor(Blue);

    GLCD_DisplayString(
        4,
        0,
        __FI,
        (unsigned char *) "Last Direction:"
    );

    GLCD_DisplayString(
        6,
        0,
        __FI,
        (unsigned char *) "NONE"
    );

#endif

    LED_Out(0);

    while (1)
    {
        joystick = get_button();

        if (joystick & KBD_UP)
        {
            last_direction = KBD_UP;
            LED_Out(1 << 0);
        }
    }
}
```

```

}#if __USE_LCD
}    GLCD_DisplayString(
        6,
        0,
        _FI,
        (unsigned char *)"UP"
    );
#endif
}

    else if (joystick & KBD_RIGHT)
    {
        last_direction = KBD_RIGHT;
        LED_Out(1 << 1);

}#if __USE_LCD
}    GLCD_DisplayString(
        6,
        0,
        _FI,
        (unsigned char *)"RIGHT"
    );
#endif
}

    else if (joystick & KBD_DOWN)
    {
        last_direction = KBD_DOWN;
        LED_Out(1 << 2);

}#if __USE_LCD
}    GLCD_DisplayString(
        6,
        0,
        _FI,
        (unsigned char *)"DOWN"
    );
#endif
}

    else if (joystick & KBD_LEFT)
    {
        last_direction = KBD_LEFT;
        LED_Out(1 << 3);

}#if __USE_LCD
}    GLCD_DisplayString(
        6,
        0,
        _FI,
        (unsigned char *)"LEFT"
    );
#endif
}

    else if (joystick & KBD_SELECT)
    {
        last_direction = KBD_SELECT;
        LED_Out(1 << 4);

}#if __USE_LCD
}    GLCD_DisplayString(
        6,
        0,
        _FI,
        (unsigned char *)"SELECT"
    );
#endif
}
}
}

```

Blinky.c

Blinky.c is the main application file shown above. It contains the main() function and controls the overall operation of the lab program. This file initializes the LEDs and joystick, reads the joystick input using get_button(), determines the direction pressed, and then updates the corresponding LED and LCD display. It also contains the option to enable or disable the LCD during debugging by defining the value of USE_LCD.

```

/*-----
 * Example header Blinky module
 * Copyright (c) 2019-2020 Arm Limited (or its affiliates). All rights reserved.
 *-----
 * Name:      Blinky.h
 * Purpose:   Blinky header
 *-----*/

/* Define to prevent recursive inclusion -----*/
#ifndef __BLINKY_H
#define __BLINKY_H

#include <stdint.h>

/* Global variables exported */
extern volatile uint8_t clock_ls;
extern uint16_t ADC_last;

#endif /* __BLINKY_H */

```

Blinky.h

This is the header file associated with Blinky.c. This file provides the appropriate declarations required to run the main application.

```

/*-----
 * Name:      IRQ.c
 * Purpose:   COE718 Lab 1 interrupt handler
 *
 * The original Blinky program used SysTick for the ADC and
 * automatic LED blinking.
 *
 * Lab 1 does not require the ADC or automatic LED blinking.
 * The joystick is read directly in Blinky.c.
 *-----*/

#include "LPC17xx.h"

/*-----
 * SysTick Interrupt Handler
 *
 * ADDED:
 * We do not need SysTick for the joystick implementation.
 * The joystick is continuously checked inside main().
 *-----*/

void SysTick_Handler(void)
{
    /*
     * Nothing is required here for Lab 1.
     */
}

```

IRQ.c

This file is supposed to contain the functions for handling the interrupts used in the project. In this particular joystick implementation, there is no need for handling interrupts, and thus the interrupt handler does nothing with regards to joystick handling.

```

#include "LPC17xx.h"
#include "LED.h"

const unsigned long led_mask[] = {
    1UL << 28,
    1UL << 29,
    1UL << 31,
    1UL << 2,
    1UL << 3,
    1UL << 4,
    1UL << 5,
    1UL << 6
};

void LED_Init(void)
{
    LPC_SC->PCONP |= (1 << 15);

    LPC_GPIO1->FIODIR |= 0xB0000000;

    LPC_GPIO2->FIODIR |= 0x0000007C;
}

void LED_On(unsigned int num)
{
    if (num < 3)
        LPC_GPIO1->FIOPIN |= led_mask[num];
    else
        LPC_GPIO2->FIOPIN |= led_mask[num];
}

void LED_Off(unsigned int num)
{
    if (num < 3)
        LPC_GPIO1->FIOPIN &= ~led_mask[num];
    else
        LPC_GPIO2->FIOPIN &= ~led_mask[num];
}

void LED_Out(unsigned int value)
{
    int i;

    for (i = 0; i < LED_NUM; i++)
    {
        if (value & (1 << i))
            LED_On(i);
        else
            LED_Off(i);
    }
}

```

LED.c

This file contains all the necessary functions for initializing and controlling the LEDs of the MCB1700 board. The main function uses functions like LED_Init() and LED_Out() from this file in order to indicate the direction of the joystick.

```

/*-----
 * Name:      LED.h
 * Purpose:   low level LED definitions
 *-----*/

#ifndef __LED_H
#define __LED_H

/* LED Definitions */
#define LED_NUM      8

extern void LED_Init(void);
extern void LED_On(unsigned int num);
extern void LED_Off(unsigned int num);
extern void LED_Out(unsigned int value);

#endif

```

LED.h

LED.h is the header file for the LED driver. It provides the declarations for the LED functions used by Blinky.c, allowing the main program to access the LED functionality implemented in LED.c.

```

#include <LPC17xx.H>
#include "KBD.h"

uint32_t KBD_val = 0;

void KBD_Init(void)
{
    LPC_SC->PCONP |= (1 << 15);

    LPC_PINCON->PINSEL3 &= ~( (3 << 8) |
                               (3 << 14) |
                               (3 << 16) |
                               (3 << 18) |
                               (3 << 20));

    LPC_GPIO1->FIODIR &= ~( (1 << 20) |
                            (1 << 23) |
                            (1 << 24) |
                            (1 << 25) |
                            (1 << 26));
}

uint32_t KBD_get(void)
{
    uint32_t kbd_val;

    kbd_val = (LPC_GPIO1->FIOPIN >> 20) & KBD_MASK;

    return kbd_val;
}

uint32_t get_button(void)
{
    uint32_t val = 0;

    val = KBD_get();

    val = (~val & KBD_MASK);

    return val;
}

```

KBD.c

KBD.c is the joystick driver for the MCB1700 board. It initializes the joystick by enabling power and assigning the GPIO pins P1.20, P1.23, P1.24, P1.25, and P1.26 for the joystick.

```

/*-----
 * Name:      KBD.h
 * Purpose:   MCB1700 low level Joystick definitions
 * Version:   V2.00
 * Note(s):   Positioning of Joystick on MCB1700
 *
 * Revised by: Anita Tino
 *-----*/

#include <stdint.h>      /* Added: defines uint32_t */

#ifndef __KBD_H
#define __KBD_H

#define KBD_SELECT      0x01
#define KBD_UP          0x08
#define KBD_RIGHT       0x10
#define KBD_DOWN        0x20
#define KBD_LEFT        0x40
#define KBD_MASK        0x79

extern uint32_t KBD_val;

extern void KBD_Init(void);
extern uint32_t KBD_get(void);
extern uint32_t get_button(void);

#endif

```

KBD.h

KBD.h is responsible for the joystick value declaration and functions. The values that indicate the direction pressed on the joystick are declared here. Those include KBD_UP, KBD_RIGHT, KBD_DOWN, KBD_LEFT, and KBD_SELECT. These declarations enable Blinky.c to recognize the direction pressed.

GLCD_SPI_LPC17xx.c (Given from lab)

GLCD_SPI_LPC17xx.c is the file that holds the LCD driver functions that allow communication between the program and the graphical LCD on the MCB1700. It has the functions necessary for initializing, clearing, configuring display options, and writing text to the LCD. The program utilizes the above functions to show the last direction of the joystick.

GLCD.h (Given from lab)

GLCD.h is the header file for the LCD driver. It provides the declarations needed by Blinky.c to access the LCD functions implemented in GLCD_SPI_LPC17xx.c.

Board Results

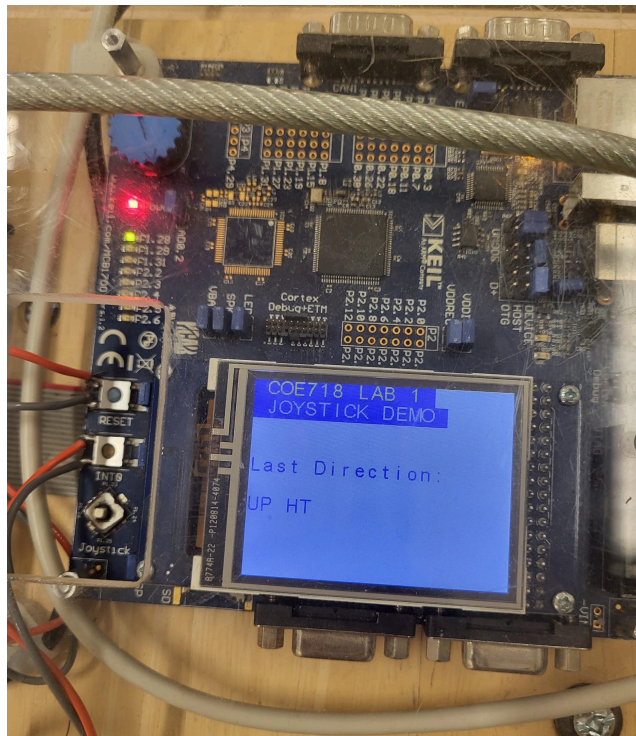


Figure 1: Result for the joystick directed upward. P1.28 LED is ON

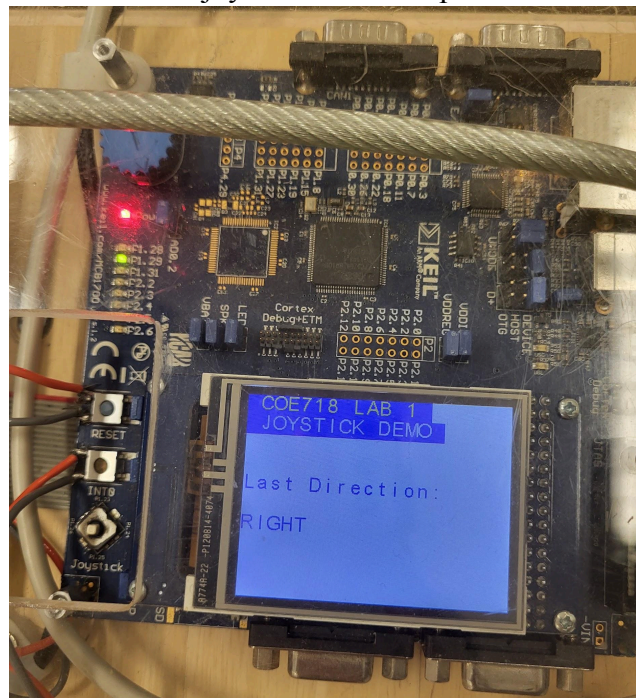


Figure 2: Result for the joystick directed right. P1.29 LED is ON



Figure 3: Result for the joystick directed left. P2.2 LED is ON

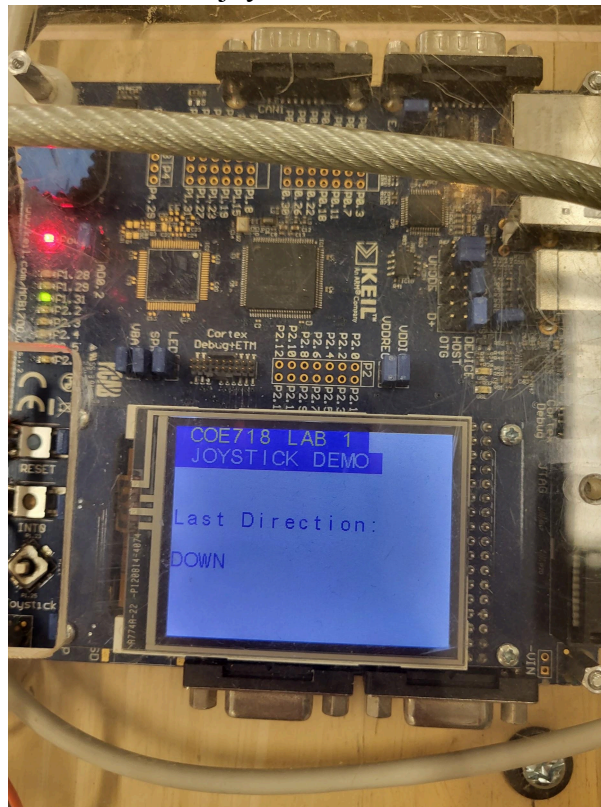


Figure 4: Result for the joystick directed downward. P1.31 LED is ON

Conclusion

In summary, this lab enabled me to gain hands-on experience in the creation of an embedded system using the Keil uVision IDE and the ARM Cortex-M3 LPC1768 processor. The lab enabled me to implement joystick input via GPIO ports and then use the direction of the joystick to control the LEDs and LCD display. The KBD.c driver was utilized to configure and take input from the joystick while the LED and GLCD drivers were responsible for providing the output. In addition, I gained experience with managing multiple C files and header files and how to configure the GPIO peripheral, as well as how embedded systems work.